

Horse Race

Learning Objective

- File I/O, csv format

Task Description

A horse race with ten horses is to be simulated. The starting list is provided in the CSV file **horses.csv**.

Race Preparation

- **ReadFromCSV**: First, the names and ages of the ten horses should be read from the provided CSV file and stored in an object array (class **Horse**). The starting numbers of the horses are assigned in the order in which they appear in the CSV file.
- **GetTip**: Output the starting list of the horses. Then the user specifies which horse they want to bet on (selection via the starting number – a number between 1 and 10). Of course, the input must be validated. Otherwise, display an error message and repeat the input.

Horse Race – Starting List:

No	Name	Age
1	Egon	6
2	Blitz	9
3	Lara	3
4	Donnerhall	5
5	Darco	10
6	Meteor	3
7	Big Ben	4
8	Goldfeuer	4
9	Silbersee	8
10	Adlerflug	7

Which horse do you want to bet on? (0 to exit)

Race Simulation

After entering the bet, the actual race starts. The horses start at position 1 and must reach position 60. (Note: Store the current positions of the horses in an integer variable of the **Horse** class.)

Your bet with starting number 5 is Darco and he is 10 years old.

Horse 1:	*
Horse 2:	*
Horse 3:	*
Horse 4:	*
Horse 5:	*

```
Horse 6:      *
Horse 7:      *
Horse 8:      *
Horse 9:      *
Horse 10:     *
```

The race proceeds in rounds. Each round includes the following steps:

- For each horse, use a random generator (**Random**) to determine whether the horse advances one position or not (probability **1:2** to advance).
- The current positions of the horses are output to the console as "*" (see screenshot). **Note:** `Console.Clear()` can be used to clear the screen.
- At the end of each round, add a `Thread.Sleep(100)` to slow down the race slightly (add `using System.Threading;` to the using section).

The race ends as soon as, after a fully completed round, at least one horse has reached position 60 (the finish). The race must not be ended in the middle of a round, but only after the position increases of all horses have been determined.

Race Evaluation

- **SortByPosition:** After the race has finished, the list of horses is first sorted by their position.
- **AssignRank:** Using this method, each horse is assigned the place / rank it achieved in the race (naturally starting with rank 1). Horses with the same position receive the same rank (*ex aequo*).
- **PrintResults:** Output of the horse list including rank, starting number, name, age, and final position reached.

```
Ihr Tipp mit der Startnummer 5 heit Darco und ist 10 Jahre alt.
Your bet with starting number 5 is Darco and he is 10 years old.
```

```
Horse 1:                                     * |
Horse 2:                                *      |
Horse 3:                                     * |
Horse 4:                                *      |
Horse 5:                        *          |
Horse 6:                                *      |
Horse 7:                        *          |
Horse 8:                                     * |
Horse 9:                                *      |
Horse 10:                        *          |
```

RACE RESULT:

=====

Rank	No	Name	Age	Position
1	8	Goldfeuer	4	60
2	3	Lara	3	57
2	9	Silbersee	8	57
4	10	Adlerflug	7	55

5	1	Egon	6	54
6	6	Meteor	3	53
7	2	Blitz	9	51
7	7	Big Ben	4	50
7	4	Donnerhall	5	50
10	5	Darco	10	48

Your bet (starting number 5), name Darco, 10 years old, achieved rank 8!

Press a key to exit ...

If you'd like, I can also:

- Reformat this as a **table or CSV**
- Explain how **ranks with ties** are calculated
- Help generate this output **programmatically in C#**

Class Horse

The data capsule for a horse is already provided in the template.

Note: The **Move** method allows the position of a horse to be increased by exactly 1.

```
public class Horse
{
    private int _number;
    private string _name;
    private int _age;
    private int _position = 1;
    private int _rank;

    public void SetNumber(int number) { ... }
    public int GetNumber() { ... }

    public void SetName(string name) { ... }
    public string GetName() { ... }

    public void SetAge(int age) { ... }
    public int GetAge() { ... }

    public void SetPosition(int position) { ... }
    public int GetPosition() { ... }

    public void SetRank(int rank) { ... }
    public int GetRank() { ... }

    public void Move()
    {
        _position++;
    }
}
```

```
} }
```

Hint

Some important hints.

User interface

N/A